Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

Python GIL, cuDF, and GPU workers with a CPU master

Setting: three ideas linked by one question: *how do we make Python data and ML work run in parallel?*

1. The GIL (Global Interpreter Lock)

What it is: a mutex in CPython (the standard Python interpreter) that allows only **one thread to execute Python bytecode at a time**, even on a machine with many cores. It exists because CPython manages memory with **reference counting**: every object has a counter of references, and without a global lock two threads updating the same counter could corrupt it. One lock is simple and makes single-threaded code fast.

Consequence: with k threads doing pure-Python CPU work, $T_{\text{threads}}(k) \approx T_1$ (speedup ≈ 1, not k), while k processes give $T_{\text{proc}}(k) \approx \frac{T_1}{k} + \text{overhead}$.

Workload	Does the GIL hurt?	Use
I/O-bound (APIs, files)	No: released while waiting for I/O	`threading`, `asyncio`
CPU-bound, pure Python loops	Yes: threads take turns	`multiprocessing`, `ProcessPoolExecutor` (one GIL per process)
CPU-bound in C / CUDA libs (NumPy, PyTorch, cuDF)	Mostly no: libraries release it while computing	Vectorize: let the library do the loop

Today: Python 3.13 added an optional **free-threaded** build without the GIL (PEP 703), officially supported in 3.14; the default build still has it.

2. cuDF (GPU DataFrames)

What it is: the dataframe library of NVIDIA **RAPIDS**. It has a **pandas-like API**, but the data lives in GPU memory (columnar, Apache Arrow format) and operations run as CUDA kernels over thousands of GPU cores. `groupby`, `merge`, `sort` and filters on millions of rows are often 10–100× faster than pandas on CPU.

```
import cudf
df = cudf.read_parquet("transactions.parquet")          # loaded into GPU memory
out = df[df.amount > 0].groupby("customer_id").amount.agg(["sum", "mean", "count"])

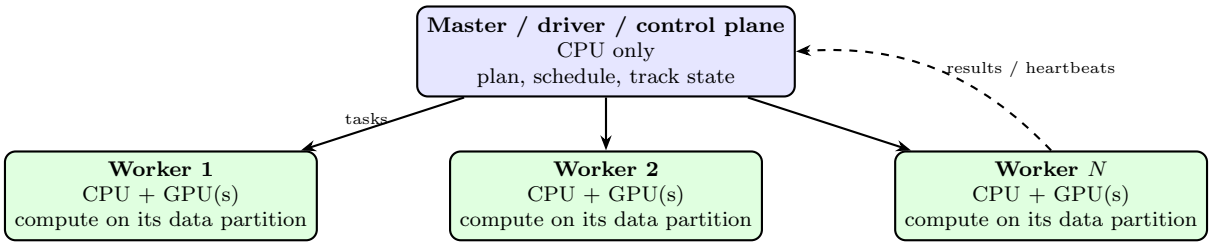
# existing pandas code, unchanged, on GPU:  python -m cudf.pandas script.py
```

Link with the GIL: the Python thread only *launches* kernels; the computation runs on the GPU outside the interpreter, so the GIL is not the bottleneck. **Limits:** data must fit in GPU memory (e.g. 16–80

GB), copying CPU ↔ GPU over PCIe is costly, and small dataframes gain little. For data larger than one GPU: **Dask-cuDF** (many GPUs) or the RAPIDS accelerator for **Spark**. Related: **cuML** (scikit-learn-like models on GPU).

3. Worker node = GPU, Master node = CPU. Why?

In a distributed cluster (Spark, Dask, Ray, Kubernetes; e.g. a CPU driver with GPU executors, or a CPU node pool plus a GPU node pool) the roles differ, and so does the hardware:



The master does control work, which is sequential and branchy → CPU:

- Builds the execution plan (DAG), splits it into tasks, assigns them to workers, monitors heartbeats, retries failures, keeps metadata (in Kubernetes: API server, scheduler, etcd).
- Logic with many `if`s, small messages and I/O, one decision at a time: a CPU (few powerful cores, large caches, branch prediction) is built for this. A GPU would sit idle: nothing to spread over thousands of cores.

The workers do the data work, which is massively parallel → GPU:

- Matrix multiplications in training and inference, and the same operation applied to millions of rows (cuDF, Spark RAPIDS): **SIMT**, thousands of simple cores running the same instruction on different data, with very high memory bandwidth (HBM, 1–3 TB/s).
- Workers still have CPUs to run the OS, load data and feed the GPU.

Why it pays off:

(a) **Cost:** GPUs are the most expensive part; one on the master would be idle almost all the time. GPUs belong where the FLOPs are.

(b) **Scaling:** add workers (N GPUs) while one modest master coordinates; worker pools autoscale without touching it.

(c) **Reliability:** isolating the master from heavy jobs prevents GPU out-of-memory errors or driver crashes from taking down the coordinator; if a worker dies, the master reschedules its tasks.

(d) **Amdahl's law:** with parallel fraction p and N workers, $S(N) = \frac{1}{(1-p)+p/N}$. GPUs on the workers accelerate the parallel part p (data and math); the serial part $1-p$ (coordination) runs on the master's CPU, where a GPU would not help.

Summary: the **GIL** serializes Python bytecode, so CPU-bound work is parallelized with processes or by delegating loops to C / CUDA libraries; **cuDF** is that delegation for dataframes, on the GPU. In a cluster the **master** coordinates (sequential → CPU) and the **workers** compute (massively parallel → GPU).